The Family Menu

Service Layer Design - Updated

Andrew Connally

03/30/2026

CHANGE/UPDATE NOTE

I've highlighted the stretch feature, beginning at Section 4. No other changes were made.

Design goal

Define every backend endpoint required to support the Meal Library, My Meal Plan, recipe overlay, and grocery list workflows for the MVP.

Overview

This document specifies a clean REST-oriented service layer for *The Family Menu*. It identifies the service endpoints required by the MVP, explains why each endpoint exists, shows example requests and responses including error cases, and maps each user interface page to the backend services it consumes.

The design assumes a Node.js/Express backend with MongoDB persistence. Meals store embedded ingredients and instructions, meal plans store references to meals, and grocery lists persist the aggregated output generated from a meal plan.

For this assignment, the Database Design document was treated as the authoritative source whenever it differed from the earlier project proposal.

---

1. Scope, Assumptions, and Service Conventions

• Base URL: /api

• Primary user flow: Busy-Mom Emma browses meals, adds meals to a saved plan, reviews recipe instructions, and views a generated grocery list.

• Editing existing meals and editing existing meal plans are included in scope.

• Authentication is included in scope. Users must register and log in to access and persist meal plans and grocery lists. All endpoints operate in the context of the authenticated user.

• The service layer returns JSON. MongoDB object identifiers are shown as ObjectId strings in examples.

• A generated grocery list is persisted as its own document so the application can read it later without recomputing every page load.

Response Envelope Convention

Success	**{ "data": { ... }, "message": "optional", "meta": { ... } }**
Client / validation error	{ "error": { "code": "VALIDATION_ERROR", "message": "...", "details": [...] } }
Common HTTP codes	200 OK, 201 Created, 400 Bad Request, 404 Not Found, 409 Conflict, 422 Unprocessable Entity

Resource Model Alignment

Resource	MongoDB collection	Why it exists	Key relationship
Meals	meals	Stores reusable recipe content shown in the meal library and recipe overlay.	Embeds ingredients and instructions.
Meal Plans	mealPlans	Stores a user's selected meals for planning and editing.	References meals by mealId.
Grocery Lists	groceryLists	Stores the generated shopping list for one meal plan.	References mealPlanId and embeds aggregated items.
Users	users	Owns meal plans and grocery lists.	Referenced by userId.

---

2. Authentication Services

Authentication services establish user identity so that meal plans and grocery lists are stored and retrieved as user-specific data rather than temporary client-side state.

ID	Method	Path	Primary page(s)	Primary data effect
E0	POST	/api/auth/register	Meal Library	Creates a new user account so meal plans and grocery

				lists can be persisted per user.
E0.1	POST	/api/auth/login	Login page	Authenticates a user and establishes an authenticated session so all subsequent requests are associated with that user.

E0. POST /api/auth/register

Example request

```
curl -X POST "https://thefamilymenu.example/api/auth/register" \
 -H "Content-Type: application/json" \
 -d '{
  "name": "Emma",
  "email": "emma@example.com",
  "password": "SecurePassword123"
 }'
```

Example success response (201)

```
{
  "data": {
    "_id": "67fe001ad2c76d4a9b4e0001",
    "email": "emma@example.com",
    "createdAt": "2026-03-23T02:00:00.000Z"
  },
  "message": "Account created successfully."
}
```

Example error response (409)

```
{
  "error": {
    "code": "EMAIL_ALREADY_EXISTS",
    "message": "An account with this email already exists."
  }
}
```

E0.1. POST /api/auth/login

Example request

```
curl -X POST "https://thefamilymenu.example/api/auth/login" \
 -H "Content-Type: application/json" \
 -d '{
  "email": "emma@example.com",
  "password": "SecurePassword123"
 }'
```

Example success response (200)

```
{
  "data": {
    "_id": "67fe001ad2c76d4a9b4e0001",
    "email": "emma@example.com"
  },
  "message": "Login successful."
```

```
}
```

Example error response (401)

```
{
  "error": {
    "code": "INVALID_CREDENTIALS",
    "message": "Email or password is incorrect."
  }
}
```

---

3. Endpoint Inventory

The table below summarizes every endpoint required to support the MVP user interface pages and the ability to create, update, and manage meals and meal plans.

ID	Method	Path	Primary page(s)	Primary data effect
E1	GET	/api/meals	Meal Library	Reads persisted data for UI rendering.
E2	GET	/api/meals/{mealId}	Meal Library, Recipe Instructions Overlay	Reads persisted data for UI rendering.
E3	POST	/api/meals	Meal Library (Create Meal modal/page)	Creates a new document or child relationship.
E4	PUT	/api/meals/{mealId}	Meal Library (Edit Meal modal/page)	Replaces or updates stored document data.
E5	GET	/api/meal-plans	My Meal Plan	Reads persisted data for UI rendering.
E6	POST	/api/meal-plans	Meal Library / My Meal Plan	Creates a new document or child relationship.
E7	GET	/api/meal-plans/{planId}	My Meal Plan	Reads persisted data for UI rendering.
E8	PUT	/api/meal-plans/{planId}	My Meal Plan (edit plan details)	Replaces or updates stored document data.
E9	POST	/api/meal-plans/{planId}/meals	Meal Library	Creates a new document or child relationship.
E10	DELETE	/api/meal-plans/{planId}/meals/{mealId}	My Meal Plan	Removes a meal reference from a saved plan.
E11	POST	/api/meal-plans/{planId}/grocery-lists/generate	My Meal Plan, My Grocery List	Creates a new document or child relationship.
E12	GET	/api/meal-plans/{planId}/grocery-lists/latest	My Grocery List	Reads persisted data for UI rendering.

4. Meals Endpoints

Meals are the reusable recipe documents shown in the Meal Library and recipe instruction overlay. They contain embedded ingredients and instructions.

E1. GET /api/meals

Purpose	**Lets Busy-Mom Emma browse the meal library. It reads meal documents so the UI can display recipe cards, summary cost, and filters without exposing database structure.**
Used by page(s)	Meal Library
Persona / data impact	This endpoint exists to support the user's goal while reading or changing persisted meal, meal plan, or grocery list data rather than relying on temporary client-side state.
Status code(s)	200 success; 400 example error

Parameters and body fields: all the inputs my endpoints accept from the client

Location	Name	Type	Notes
Query	search	string	Optional text matched against title, description, and tags.
Query	tag	string	Optional tag filter such as kid-friendly.
Query	sort	string	Optional sort value: title, costAsc, costDesc, newest.

Example request

```
curl -X GET "https://thefamilymenu.example/api/meals?search=pasta&tag=kid-friendly&sort=title"
```

Example success response (200)

```
{
  "data": [
    {
      "_id": "67fe01c2d2c76d4a9b4e1001",
      "title": "Creamy Chicken Pasta",
      "description": "A quick weeknight dinner for busy families.",
      "imageUrl": "/images/creamy-chicken-pasta.jpg",
      "estimatedMealCost": 18.5,
      "tags": [
        "kid-friendly",
        "quick"
      ]
    }
  ],
  "meta": {
    "count": 1,
    "filters": {
      "search": "pasta",
```

```
      "tag": "kid-friendly",
      "sort": "title"
    }
  }
}
```

Example error response (400)

```
{
  "error": {
    "code": "INVALID_QUERY_PARAMETER",
    "message": "sort must be one of: title, costAsc, costDesc, newest."
  }
}
```

E2. GET /api/meals/{mealId}

Purpose	**Loads one full meal document so the user can inspect ingredients and step-by-step instructions before cooking or while editing the recipe.**
Used by page(s)	Meal Library, Recipe Instructions Overlay
Persona / data impact	This endpoint exists to support the user's goal while reading or changing persisted meal, meal plan, or grocery list data rather than relying on temporary client-side state.
Status code(s)	200 success; 404 example error

Parameters and body fields: all the inputs my endpoints accept from the client

Location	Name	Type	Notes
Path	mealId	ObjectId	Required meal identifier.

Example request

```
curl -X GET "https://thefamilymenu.example/api/meals/67fe01c2d2c76d4a9b4e1001"
```

Example success response (200)

```
{
  "data": {
    "_id": "67fe01c2d2c76d4a9b4e1001",
    "title": "Creamy Chicken Pasta",
    "description": "A quick weeknight dinner for busy families.",
    "imageUrl": "/images/creamy-chicken-pasta.jpg",
    "estimatedMealCost": 18.5,
    "ingredients": [
      {
        "ingredientId": "ING-CHICKEN-BREAST",
        "name": "Chicken breast",
        "quantity": 2,
        "unit": "lb",
        "estimatedUnitCost": 5.5
      },
      {
        "ingredientId": "ING-PASTA-PENNE",
        "name": "Penne pasta",
        "quantity": 1,
```

```
        "unit": "box",
        "estimatedUnitCost": 1.75
      }
    ],
    "instructions": [
      "Boil pasta.",
      "Cook chicken.",
      "Stir in sauce and combine."
    ],
    "tags": [
      "kid-friendly",
      "quick"
    ],
    "createdAt": "2026-03-20T18:30:00.000Z",
    "updatedAt": "2026-03-22T16:10:00.000Z"
  }
}
```

Example error response (404)

```
{
  "error": {
    "code": "MEAL_NOT_FOUND",
    "message": "No meal exists for mealId 67fe01c2d2c76d4a9b4e1001."
  }
}
```

E3. POST /api/meals

Purpose	**Creates a new meal document so the household or application maintainer can expand the meal library with a reusable recipe.**
Used by page(s)	Meal Library (Create Meal modal/page)
Persona / data impact	This endpoint exists to support the user's goal while reading or changing persisted meal, meal plan, or grocery list data rather than relying on temporary client-side state.
Status code(s)	201 success; 422 example error

Parameters and body fields: all the inputs my endpoints accept from the client

Location	Name	Type	Notes
Body	title	string	Required. User-facing meal title.
Body	ingredients	array	Required. Embedded ingredient objects.
Body	instructions	array	Required. Ordered recipe steps.
Body	tags	array<string>	Optional classification tags.

Example request

```
curl -X POST "https://thefamilymenu.example/api/meals" \
  -H "Content-Type: application/json" \
  -d '{
```

```
    "title": "Sheet Pan Sausage and Veggies",
    "description": "One-pan dinner with minimal cleanup.",
    "imageUrl": "/images/sheet-pan.jpg",
    "ingredients": [
      {"ingredientId":"ING-SAUSAGE","name":"Italian
sausage","quantity":2,"unit":"lb","estimatedUnitCost":4.75},

{"ingredientId":"ING-BROCCOLI","name":"Broccoli","quantity":2,"unit":"head","estimatedUnitCost":2.25
}
    ],
    "instructions": ["Preheat oven to 425 F.","Slice sausage and vegetables.","Roast for 25
minutes."],
    "tags": ["quick","kid-friendly"]
  }'
```

Example success response (201)

```
{
  "data": {
    "_id": "67fe01c2d2c76d4a9b4e1099",
    "title": "Sheet Pan Sausage and Veggies",
    "estimatedMealCost": 14.0,
    "createdAt": "2026-03-23T01:00:00.000Z",
    "updatedAt": "2026-03-23T01:00:00.000Z"
  },
  "message": "Meal created successfully."
}
```

Example error response (422)

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "ingredients[0].unit is required.",
    "details": [
      {
        "field": "ingredients[0].unit",
        "issue": "missing"
      }
    ]
  }
}
```

E4. PUT /api/meals/{mealId}

Purpose	**Updates an existing meal so future meal plans and future grocery lists use the current recipe data stored in the meals collection.**
Used by page(s)	Meal Library (Edit Meal modal/page)
Persona / data impact	This endpoint exists to support the user's goal while reading or changing persisted meal, meal plan, or grocery list data rather than relying on temporary client-side state.
Status code(s)	200 success; 404 example error

Parameters and body fields: all the inputs my endpoints accept from the client

Location	Name	Type	Notes
Path	mealId	ObjectId	Required meal identifier.
Body	title / description / imageUrl	mixed	Mutable top-level meal fields.
Body	ingredients	array	Optional full replacement of embedded ingredient array.
Body	instructions	array	Optional full replacement of instruction list.

Example request

```
curl -X PUT "https://thefamilymenu.example/api/meals/67fe01c2d2c76d4a9b4e1001" \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Creamy Chicken Pasta",
    "description": "Updated to include more vegetables.",
    "ingredients": [
      {"ingredientId":"ING-CHICKEN-BREAST","name":"Chicken
breast","quantity":2,"unit":"lb","estimatedUnitCost":5.5},

{"ingredientId":"ING-SPINACH","name":"Spinach","quantity":1,"unit":"bag","estimatedUnitCost":2.99}
    ],
    "instructions": ["Boil pasta.","Cook chicken.","Add spinach and sauce.","Combine and serve."],
    "tags": ["kid-friendly","quick"]
  }'
```

Example success response (200)

```
{
  "data": {
    "_id": "67fe01c2d2c76d4a9b4e1001",
    "updatedAt": "2026-03-23T01:20:00.000Z"
  },
  "message": "Meal updated successfully."
}
```

Example error response (404)

```
{
  "error": {
    "code": "MEAL_NOT_FOUND",
    "message": "Cannot update meal because the mealId does not exist."
  }
}
```

---

5. Meal Plans Endpoints

Meal plans persist a user's selected meals and their current planning state. They are the bridge between browsing recipes and generating a grocery list.

E5. GET /api/meal-plans

Purpose	**Retrieves the current user's meal plans. The UI uses status=draft to load the editable plan that Emma is actively building.**
Used by page(s)	My Meal Plan
Persona / data impact	This endpoint exists to support the user's goal while reading or changing persisted meal, meal plan, or grocery list data rather than relying on temporary client-side state.
Status code(s)	200 success; 400 example error

Parameters and body fields: all the inputs my endpoints accept from the client

Location	Name	Type	Notes
Query	userId	ObjectId	Required for ownership scoping in this MVP design.
Query	status	string	Optional status filter: draft, finalized, archived.

Example request

```
curl -X GET
"https://thefamilymenu.example/api/meal-plans?userId=67fe001ad2c76d4a9b4e0001&status=draft"
```

Example success response (200)

```
{
  "data": [
    {
      "_id": "67fe0a70d2c76d4a9b4e2001",
      "userId": "67fe001ad2c76d4a9b4e0001",
      "status": "draft",
      "estimatedTotalCost": 36.75,
      "mealCount": 2,
      "createdAt": "2026-03-22T19:00:00.000Z",
      "updatedAt": "2026-03-23T01:22:00.000Z"
    }
  ],
  "meta": {
    "count": 1
  }
}
```

Example error response (400)

```
{
  "error": {
    "code": "USER_ID_REQUIRED",
    "message": "userId query parameter is required."
  }
}
```

E6. POST /api/meal-plans

Purpose	**Creates a new meal plan document for the authenticated household user so selected meals can be persisted over time instead of only living in browser state.**
Used by page(s)	Meal Library / My Meal Plan
Persona / data impact	This endpoint exists to support the user's goal while reading or changing persisted meal, meal plan, or grocery list data rather than relying on temporary client-side state.
Status code(s)	201 success; 409 example error

Parameters and body fields: all the inputs my endpoints accept from the client

Location	Name	Type	Notes
Body	userId	ObjectId	Required plan owner.
Body	status	string	Optional initial status; defaults to draft.
Body	meals	array	Optional initial meal references.

Example request

```
curl -X POST "https://thefamilymenu.example/api/meal-plans" \
  -H "Content-Type: application/json" \
  -d '{"userId":"67fe001ad2c76d4a9b4e0001","status":"draft","meals":[]}'
```

Example success response (201)

```
{
  "data": {
    "_id": "67fe0a70d2c76d4a9b4e2001",
    "userId": "67fe001ad2c76d4a9b4e0001",
    "status": "draft",
    "meals": [],
    "estimatedTotalCost": 0
  },
  "message": "Meal plan created successfully."
}
```

Example error response (409)

```
{
  "error": {
    "code": "DRAFT_PLAN_ALREADY_EXISTS",
    "message": "This user already has an active draft meal plan."
  }
}
```

E7. GET /api/meal-plans/{planId}

Purpose	**Loads one meal plan and can expand meal references into meal summaries so the page can render selected meals, costs, and recipe actions in a single request.**
Used by page(s)	My Meal Plan
Persona / data impact	This endpoint exists to support the user's goal while reading or changing persisted meal, meal plan, or grocery list data rather than relying on temporary client-side state.
Status code(s)	200 success; 404 example error

Parameters and body fields: all the inputs my endpoints accept from the client

Location	Name	Type	Notes
Path	planId	ObjectId	Required meal plan identifier.
Query	includeMeals	boolean	When true, embed read-only meal summaries in the response.

Example request

```
curl -X GET
"https://thefamilymenu.example/api/meal-plans/67fe0a70d2c76d4a9b4e2001?includeMeals=true"
```

Example success response (200)


```
{
  "data": {
    "_id": "67fe0a70d2c76d4a9b4e2001",
    "userId": "67fe001ad2c76d4a9b4e0001",
    "status": "draft",
    "estimatedTotalCost": 36.75,
    "meals": [
      {
        "mealId": "67fe01c2d2c76d4a9b4e1001",
        "servingsOverride": null,
        "meal": {
          "title": "Creamy Chicken Pasta",
          "estimatedMealCost": 18.5,
          "imageUrl": "/images/creamy-chicken-pasta.jpg"
        }
      },
      {
        "mealId": "67fe01c2d2c76d4a9b4e1002",
        "servingsOverride": null,
        "meal": {
          "title": "Taco Rice Bowls",
          "estimatedMealCost": 18.25,
          "imageUrl": "/images/taco-rice-bowls.jpg"
        }
      }
    ],
    "updatedAt": "2026-03-23T01:22:00.000Z"
  }
}
```

Example error response (404)

```
{
  "error": {
    "code": "MEAL_PLAN_NOT_FOUND",
    "message": "No meal plan exists for the supplied planId."
  }
}
```

E8. PUT /api/meal-plans/{planId}

Purpose	Edits high-level meal plan data such as status or servings overrides so the saved plan reflects the user's latest planning decisions.
Used by page(s)	My Meal Plan (edit plan details)
Persona / data impact	This endpoint exists to support the user's goal while reading or changing persisted meal, meal plan, or grocery list data rather than relying on temporary client-side state.
Status code(s)	200 success; 422 example error

Parameters and body fields: all the inputs my endpoints accept from the client

Location	Name	Type	Notes
Path	planId	ObjectId	Required meal plan identifier.
Body	status	string	Optional replacement status.
Body	meals	array	Optional full replacement of referenced meals for bulk editing.

Example request

```
curl -X PUT "https://thefamilymenu.example/api/meal-plans/67fe0a70d2c76d4a9b4e2001" \
  -H "Content-Type: application/json" \
  -d '{
    "status":"finalized",
    "meals":[
      {"mealId":"67fe01c2d2c76d4a9b4e1001","servingsOverride":6},
      {"mealId":"67fe01c2d2c76d4a9b4e1002","servingsOverride":4}
    ]
  }'
```

Example success response (200)

```
{
  "data": {
    "_id": "67fe0a70d2c76d4a9b4e2001",
    "status": "finalized",
    "estimatedTotalCost": 36.75,
    "updatedAt": "2026-03-23T01:30:00.000Z"
  },
  "message": "Meal plan updated successfully."
}
```

Example error response (422)

```
{
  "error": {
    "code": "INVALID_PLAN_STATUS",
    "message": "status must be one of: draft, finalized, archived."
  }
}
```

E9. POST /api/meal-plans/{planId}/meals

Purpose	**Adds one meal reference to the selected meal plan. This is the key write operation behind the Add to Meal Plan button in the wireframes.**
Used by page(s)	Meal Library
Persona / data impact	This endpoint exists to support the user's goal while reading or changing persisted meal, meal plan, or grocery list data rather than relying on temporary client-side state.
Status code(s)	201 success; 409 example error

Parameters and body fields: all the inputs my endpoints accept from the client

Location	Name	Type	Notes
Path	planId	ObjectId	Required meal plan identifier.
Body	mealId	ObjectId	Required meal identifier to add.
[redacted]	[redacted]	[redacted]	[redacted]

Example request

```
curl -X POST "https://thefamilymenu.example/api/meal-plans/67fe0a70d2c76d4a9b4e2001/meals" \
  -H "Content-Type: application/json" \
  [redacted]
```

Example success response (201)

```
{
  "data": {
    "planId": "67fe0a70d2c76d4a9b4e2001",
    "mealCount": 3,
    "estimatedTotalCost": 55.25
  },
  "message": "Meal added to meal plan successfully."
}
```

Example error response (409)

```
{
  "error": {
    "code": "MEAL_ALREADY_IN_PLAN",
    "message": "This meal has already been added to the selected meal plan."
  }
}
```

E10. DELETE /api/meal-plans/{planId}/meals/{mealId}

Purpose	**Removes a selected meal from the saved plan so the plan's meal list, cost total, and future grocery list all reflect the user's updated selection.**
Used by page(s)	My Meal Plan
Persona / data impact	This endpoint exists to support the user's goal while reading or changing persisted meal, meal plan, or grocery list data rather than relying on temporary client-side state.
Status code(s)	200 success; 404 example error

Parameters and body fields: all the inputs my endpoints accept from the client

Location	Name	Type	Notes
Path	planId	ObjectId	Required meal plan identifier.
Path	mealId	ObjectId	Required meal identifier to remove.

Example request

```
curl -X DELETE
"https://thefamilymenu.example/api/meal-plans/67fe0a70d2c76d4a9b4e2001/meals/67fe01c2d2c76d4a9b4e100
1"
```

Example success response (200)

```
{
  "data": {
    "planId": "67fe0a70d2c76d4a9b4e2001",
    "mealCount": 1,
    "estimatedTotalCost": 18.25
  },
  "message": "Meal removed from meal plan successfully."
}
```

Example error response (404)

```
{
  "error": {
    "code": "MEAL_NOT_IN_PLAN",
    "message": "The requested meal is not present in the specified meal plan."
  }
}
```

---

6. Grocery Lists Endpoints

Grocery list services execute the core business logic of the application by aggregating ingredients across the selected meals in a plan and persisting the output.

E11. POST /api/meal-plans/{planId}/grocery-lists/generate

Purpose	**Runs the core business logic: load referenced meals, aggregate duplicate ingredients by ingredientId, calculate quantities, estimate line-item cost, and persist the generated grocery list.**
Used by page(s)	My Meal Plan, My Grocery List
Persona / data impact	This endpoint exists to support the user's goal while reading or changing persisted meal, meal plan, or grocery list data rather than relying on temporary client-side state.
Status code(s)	201 success; 400 example error

Parameters and body fields: all the inputs my endpoints accept from the client

Location	Name	Type	Notes
Path	planId	ObjectId	Required meal plan identifier.
Body	forceRegenerate	boolean	Optional. When true, replace an existing generated list.

Example request

```
curl -X POST
"https://thefamilymenu.example/api/meal-plans/67fe0a70d2c76d4a9b4e2001/grocery-lists/generate" \
  -H "Content-Type: application/json" \
  -d '{"forceRegenerate":true}'
```

Example success response (201)

```
{
  "data": {
    "_id": "67fe0c8cd2c76d4a9b4e3001",
    "mealPlanId": "67fe0a70d2c76d4a9b4e2001",
    "estimatedTotalCost": 32.45,
    "items": [
      {
        "ingredientId": "ING-BROCCOLI",
        "name": "Broccoli",
        "totalQuantity": 3,
        "unit": "head",
        "estimatedLineCost": 6.75,
        "sourceMealIds": [
          "67fe01c2d2c76d4a9b4e1001",
          "67fe01c2d2c76d4a9b4e1002"
        ]
      }
    ],
    "generatedAt": "2026-03-23T01:34:00.000Z",
    "updatedAt": "2026-03-23T01:34:00.000Z"
  },
  "message": "Grocery list generated successfully."
}
```

Example error response (400)

```
{
  "error": {
    "code": "EMPTY_MEAL_PLAN",
    "message": "Cannot generate a grocery list for a meal plan with zero meals."
  }
}
```

E12. GET /api/meal-plans/{planId}/grocery-lists/latest

Purpose	**Returns the most recent grocery list already generated for the selected plan so the grocery page can render quickly without recomputing the aggregation every time.**
Used by page(s)	My Grocery List
Persona / data impact	This endpoint exists to support the user's goal while reading or changing persisted meal, meal plan, or grocery list data rather than relying on temporary client-side state.
Status code(s)	200 success; 404 example error

Parameters and body fields: all the inputs my endpoints accept from the client

Location	Name	Type	Notes
Path	planId	ObjectId	Required meal plan identifier.

Example request

```
curl -X GET
"https://thefamilymenu.example/api/meal-plans/67fe0a70d2c76d4a9b4e2001/grocery-lists/latest"
```

Example success response (200)

```
{
  "data": {
    "_id": "67fe0c8cd2c76d4a9b4e3001",
    "mealPlanId": "67fe0a70d2c76d4a9b4e2001",
    "estimatedTotalCost": 32.45,
    "items": [
      {
        "ingredientId": "ING-BROCCOLI",
        "name": "Broccoli",
        "totalQuantity": 3,
        "unit": "head",
        "estimatedLineCost": 6.75,
        "sourceMealIds": [
          "67fe01c2d2c76d4a9b4e1001",
          "67fe01c2d2c76d4a9b4e1002"
        ]
      }
    ],
    "generatedAt": "2026-03-23T01:34:00.000Z"
  }
}
```

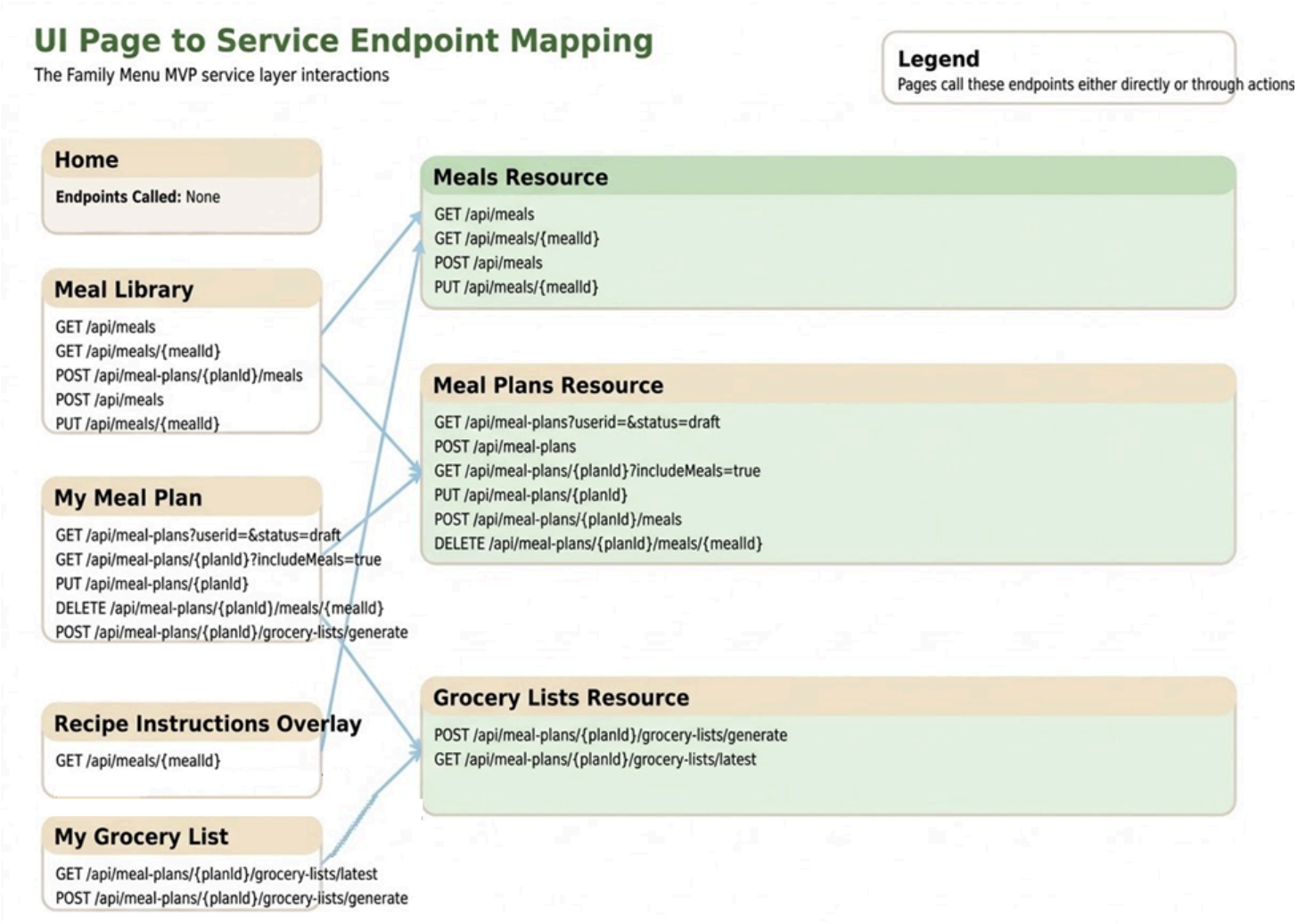
Example error response (404)

```
{
  "error": {
    "code": "GROCERY_LIST_NOT_FOUND",
    "message": "No generated grocery list exists yet for this meal plan."
  }
}
```

---

7. UI to Endpoint Diagram

The diagram below shows which user interface pages or overlays communicate with which backend resources. Home is informational only; the other pages all depend on the service layer.



Page-to-endpoint matrix

UI page	Endpoint(s) called	Why the call is needed
Home	None	Static landing page based on the provided wireframe.
Meal Library	GET /api/meals; GET /api/meals/{mealId}; POST /api/meal-plans/{planId}/meals; POST /api/meals; PUT /api/meals/{mealId}	Displays recipe cards, supports viewing full meal details, adding a meal to the plan, and creating/editing meal records.
My Meal Plan	GET /api/meal-plans?userId=&status=draft; GET /api/meal-plans/{planId}?includeMeals=true; PUT /api/meal-plans/{planId}; DELETE /api/meal-plans/{planId}/meals/{mealId}; POST /api/meal-plans/{planId}/grocery-lists/generate	Loads the saved plan, supports plan edits, allows meal removal, and can regenerate the grocery list after plan changes.
Recipe Instructions Overlay	GET /api/meals/{mealId}	Shows the instruction steps for a selected meal from the current plan.
My Grocery List	GET /api/meal-plans/{planId}/grocery-lists/latest; POST /api/meal-plans/{planId}/grocery-lists/generate	Loads the persisted grocery list or generates one if the user has not created it yet or if the plan changed.

---

8. Implementation Notes and REST Justification

• RESTful nouns are used for the primary resources: meals, meal-plans, and grocery-lists. HTTP verbs communicate the action instead of embedding verbs in most paths.

• Nested routes are used only where the child resource depends on its parent for context, such as adding a meal reference to a specific meal plan or generating the grocery list for a specific plan.

• GET endpoints do not mutate server state. POST creates documents or relationships. PUT updates existing documents. DELETE removes a meal reference from a plan.

• Grocery list generation is domain logic, not a simple CRUD create of arbitrary client-provided data. The server derives the document from the meal plan and referenced meals. Even when only one meal is selected, the backend still performs application logic by creating and persisting a user-specific grocery list derived from that saved plan.

• Persisting the generated grocery list matches the database design and prevents unnecessary recalculation on every page load.

• Editing a meal changes future grocery list generations because meal plans reference current meal documents. Already-generated grocery lists remain unchanged until regeneration, which is consistent with the database design document.

9. Conclusion

This service layer design is complete enough to implement the MVP from the wireframes, storyboard flow, style guide, and database design. It defines the necessary interface between the frontend and MongoDB-backed backend services while keeping the contract clear, scalable, and consistent.